

class named `date`. Objects of the `datetime.date` class (also simply called `datetime.date` objects or `date` objects) represent a specific date. Enter the following in the interactive shell to create an object of the `datetime.date` class:

```
>>> import datetime
>>> birthday = datetime.date(1999, 10, 31) # Pass the year, month, and day.
>>> birthday.year
1999
>>> birthday.month
10
>>> birthday.day
31
>>> birthday.weekday() # weekday() is a method; note the parentheses.
6
```

Attributes are variables associated with objects. The call to `datetime.date()` creates a new `date` object, initialized with the arguments 1999, 10, 31 so the object represents the date October 31, 1999. We assign these arguments as the `date` class's `year`, `month`, and `day` attributes, which all `date` objects have.

With this information, the class's `weekday()` method can calculate the day of the week. In this example, it returns 6 for Sunday, because according to Python's online documentation, the return value of `weekday()` is an integer that starts at 0 for Monday and goes to 6 for Sunday. The documentation lists several other methods that objects of the `date` class have. Even though the `date` object contains multiple attributes and methods, it's still a single object that you can store in a variable, such as `birthday` in this example.

Creating a Simple Class: WizCoin

Let's create a `WizCoin` class, which represents a number of coins in a fictional wizard currency. In this currency, the denominations are knuts, sickles (worth 29 knuts), and galleons (worth 17 sickles or 493 knuts). Keep in mind that the objects in the `WizCoin` class represent a quantity of coins, not an amount of money. For example, it will inform you that you're holding five quarters and one dime rather than \$1.35.

In a new file named `wizcoin.py`, enter the following code to create the `WizCoin` class. Note that the `__init__` method name has two underscores before